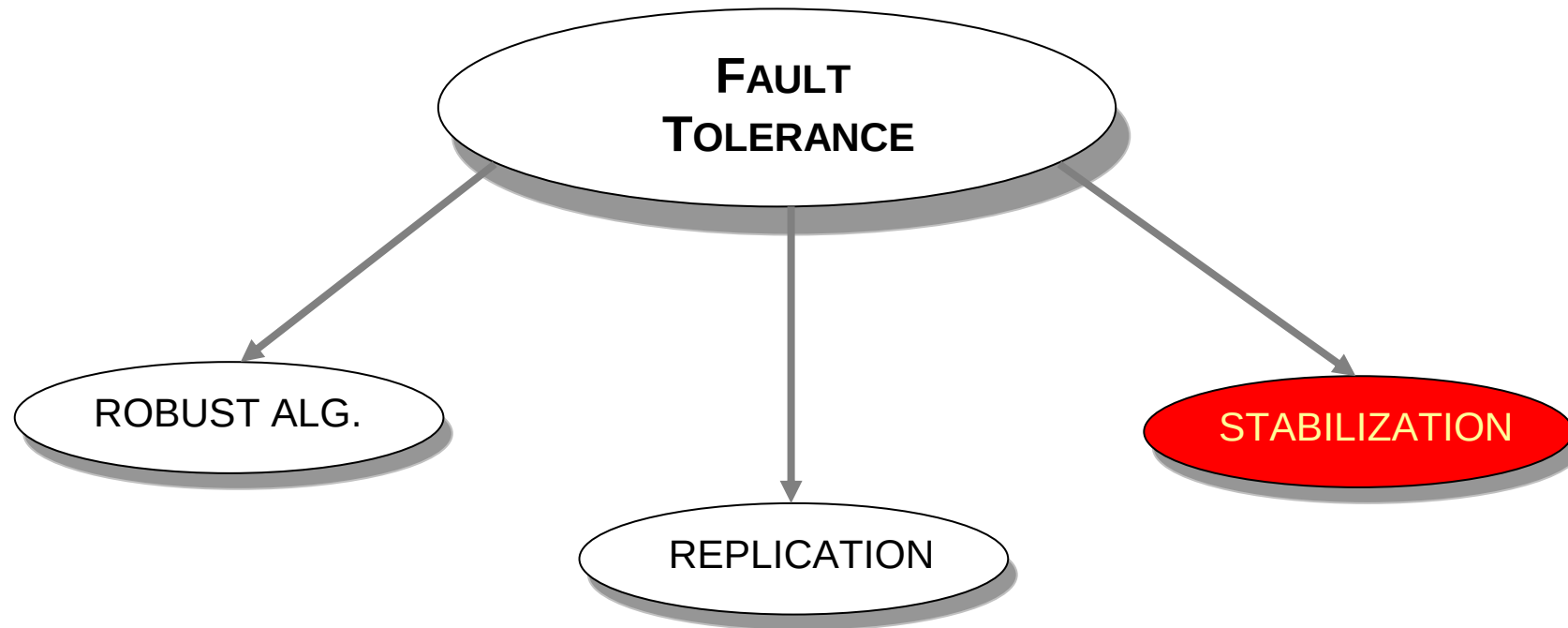


MAIN TOPICS OF THE COURSE





SELF-STABILIZATION

Self-stabilization of a system guarantees that regardless of its current state, the system will converge to a legal state in a finite number of steps.



Definition 1. A **system** is a pair $S = (\mathbb{C}, \rightarrow)$, where $\mathbb{C}$ is a set of states of the system S and $\rightarrow$ is a binary transition relation on $\mathbb{C}$. A **computation** of S is a non-empty sequence $(\Sigma_0, \Sigma_1, \Sigma_2, \dots)$ such that for all $i \geq 0$: $\Sigma_i \in \mathbb{C}$ and $\Sigma_i \rightarrow \Sigma_{i+1}$.

In the distributed systems every above mentioned Σ_i express a global state. Thus we must remember that a global state is a concatenation of the local states of every process and the contents of every communication channel.

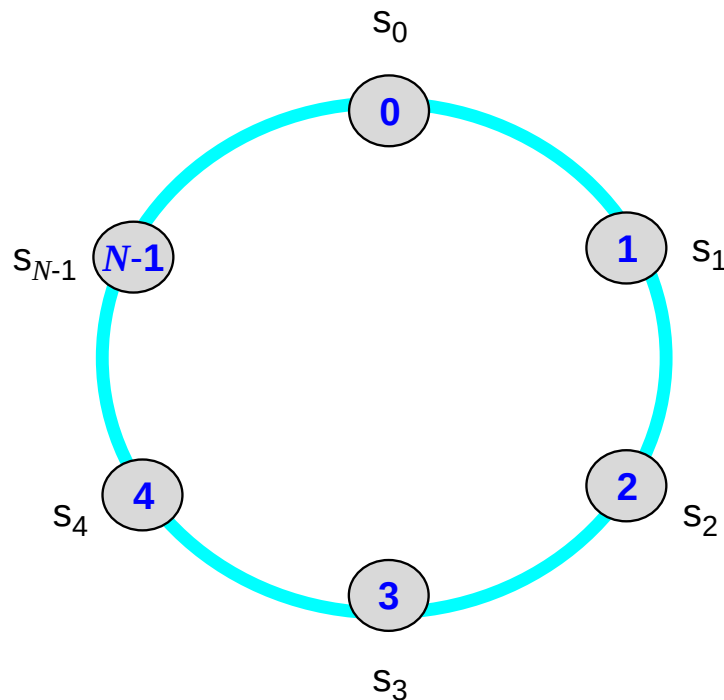
Definition 2. System S is **self-stabilizing** if there exists a subset $\mathbb{L} \subset \mathbb{C}$ of all legal states, such that:

- every computation starting in a state from $\mathbb{L}$ is correct (**correctness**)
- and
- every computation of S has a state from $\mathbb{L}$ (**convergence**).

Subset $\mathbb{L}$ is in strict relation with a given problem. Legal states are related to the notion of correctness of a given algorithm (but we do not include any formal definition of legal states). Sometimes we explicitly add that subset $\mathbb{L}$ must be closed, i.e. every (non-faulty) transition from a legal state put the system into another legal state.

FIRST EXAMPLE

Dijkstra's mutual exclusion (1974)



- N -processes ring
- one distinguished process P_0
- $P_1 \div P_{N-1}$ processes are equal and behave uniformly
- s_i = current local state of a given process P_i
- every process P_i knows the state of its left neighbor (s_{i-1} for $1 \leq i \leq N-1$ and s_{N-1} for P_0) – instantaneous communication

Notion of *privilege*

- When a process has the *privilege*, it is authorized to *make a move* (enter the critical section)
- The system satisfies the following properties:
 1. There must be at least one privilege in the system.
 2. During an infinite time every process should be able to receive a privilege infinitely many times.
- A legal (global) state must satisfy:
 3. There is only one privilege in the system.

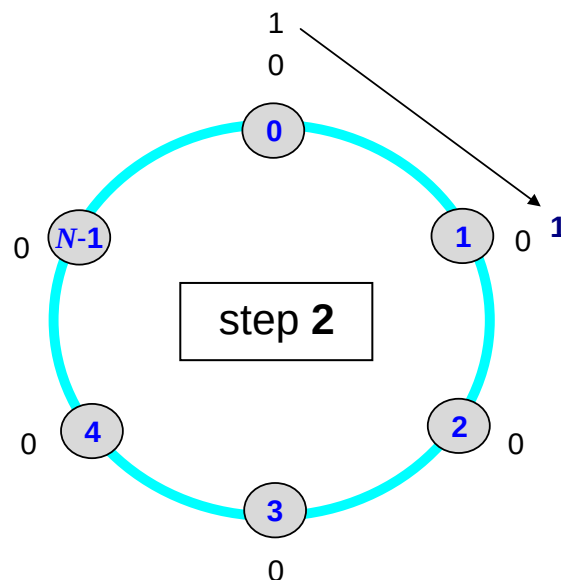
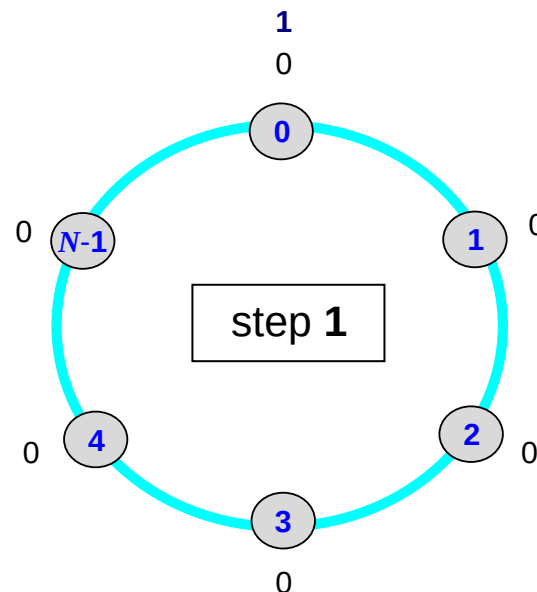
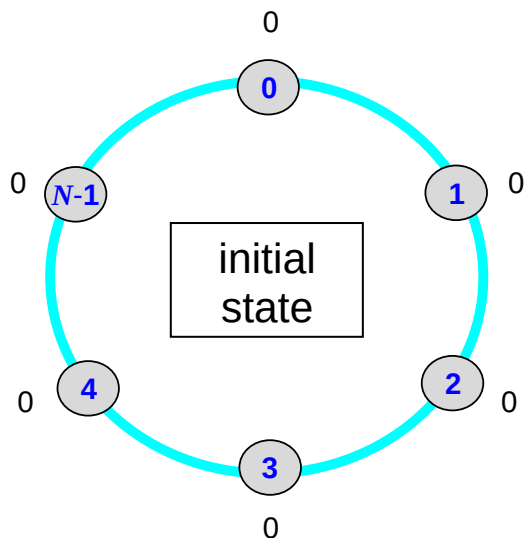
for P_0 : if $s_0 = s_{N-1}$ then P_0 has the privilege and $s_0 = (s_0 + 1) \bmod K$

for P_i : if $s_i \neq s_{i-1}$ then P_i has the privilege and $s_i = s_{i-1}$; $1 \leq i \leq N-1$

DIJKSTRA'S MUTUAL EXCLUSION ALGORITHM

(3)

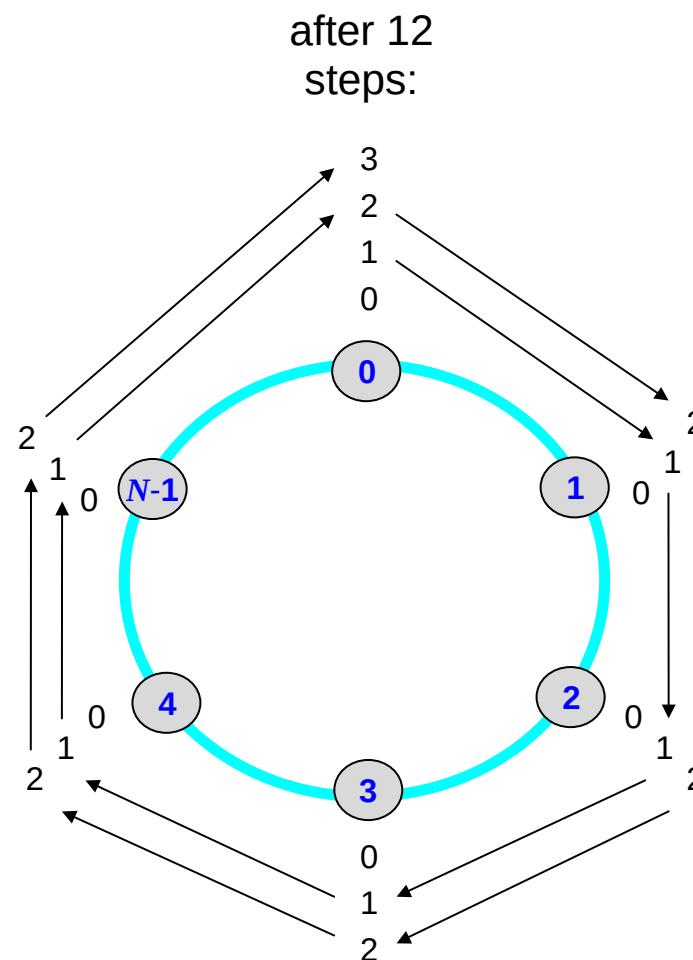
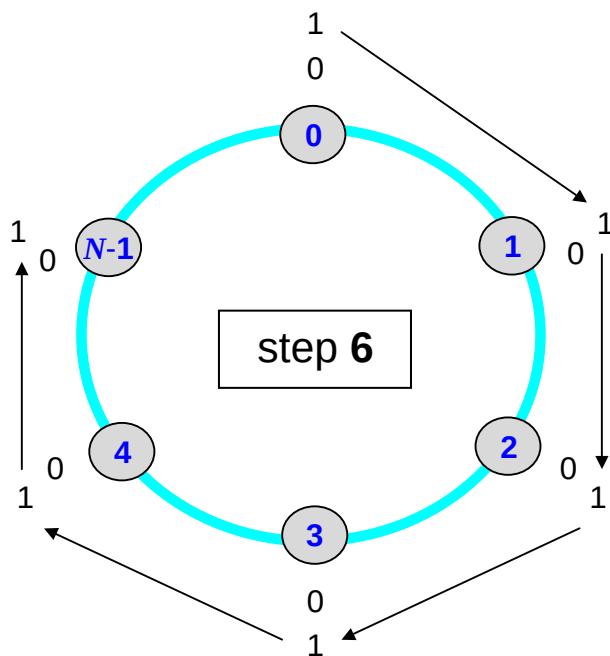
INSTYTUT
INFORMATYKI



DIJKSTRA'S MUTUAL EXCLUSION ALGORITHM

(4)

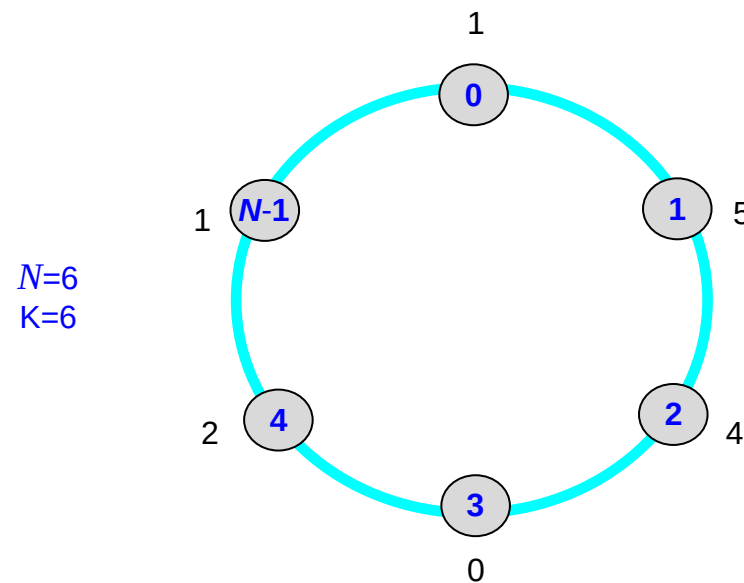
INSTYTUT
INFORMATYKI





Possible failure scenario:

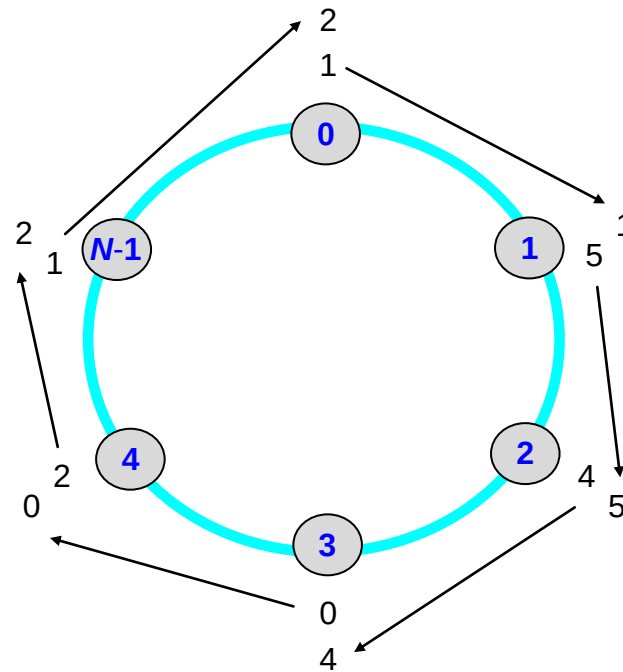
Let's assume a transient failure has put the system into some illegal state:



DIJKSTRA'S MUTUAL EXCLUSION ALGORITHM

(6)

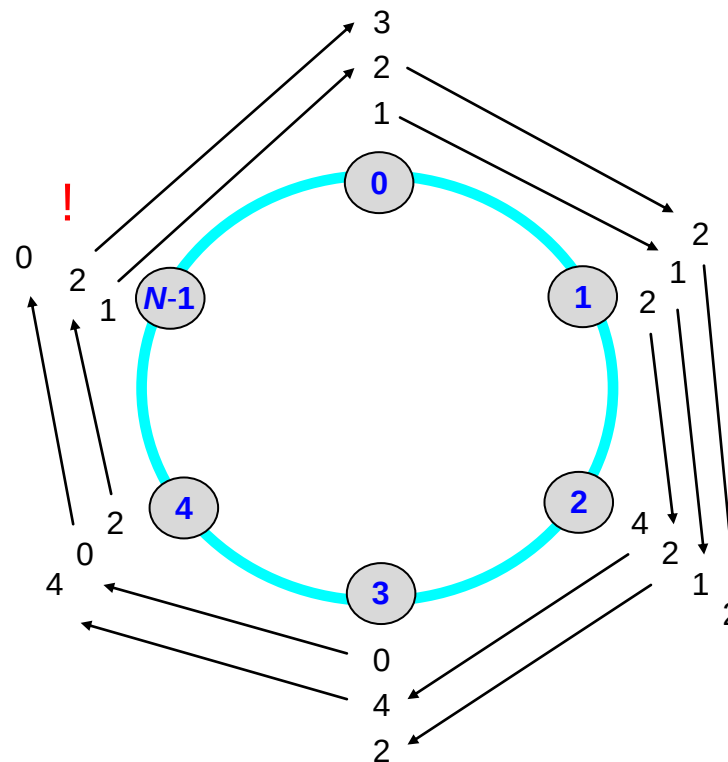
INSTYTUT
INFORMATYKI



DIJKSTRA'S MUTUAL EXCLUSION ALGORITHM

(7)

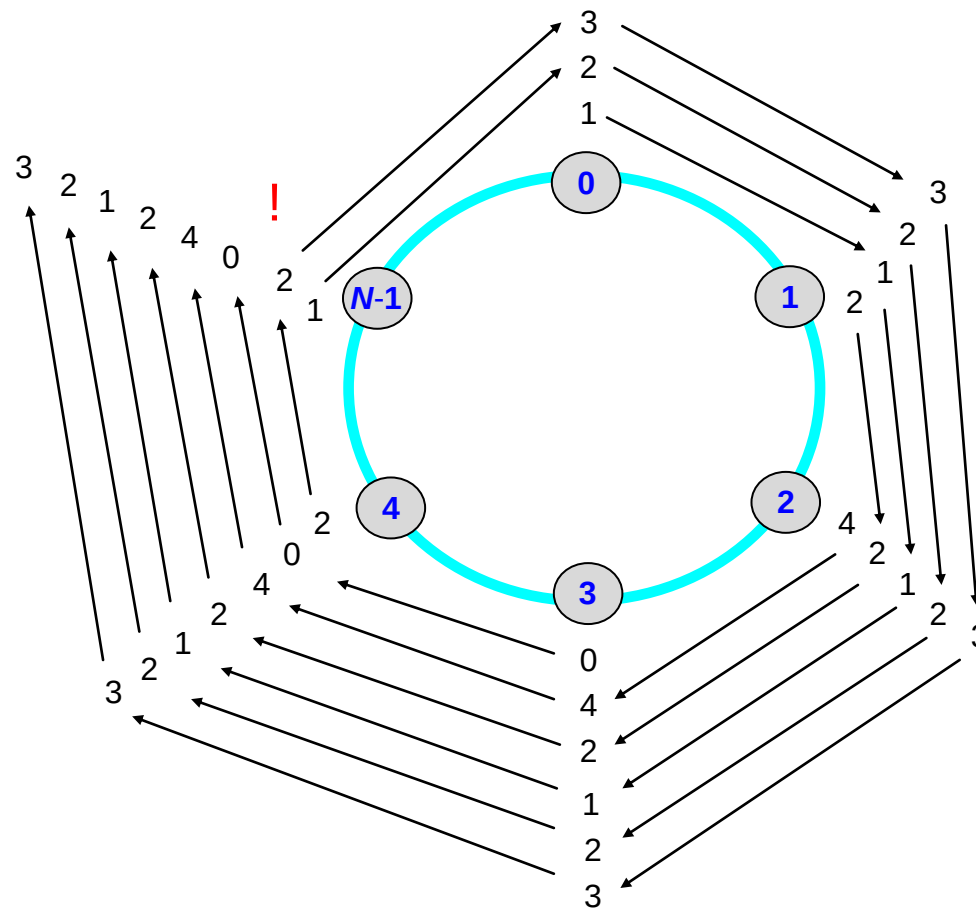
INSTYTUT
INFORMATYKI



DIJKSTRA'S MUTUAL EXCLUSION ALGORITHM

(8)

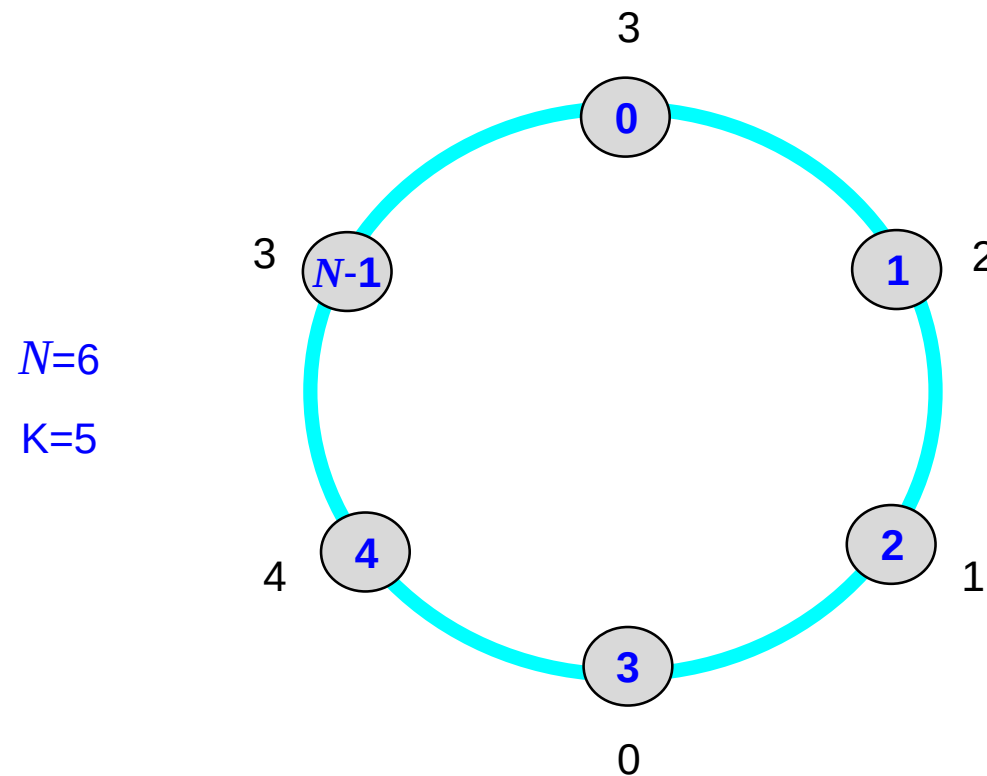
INSTYTUT
INFORMATYKI



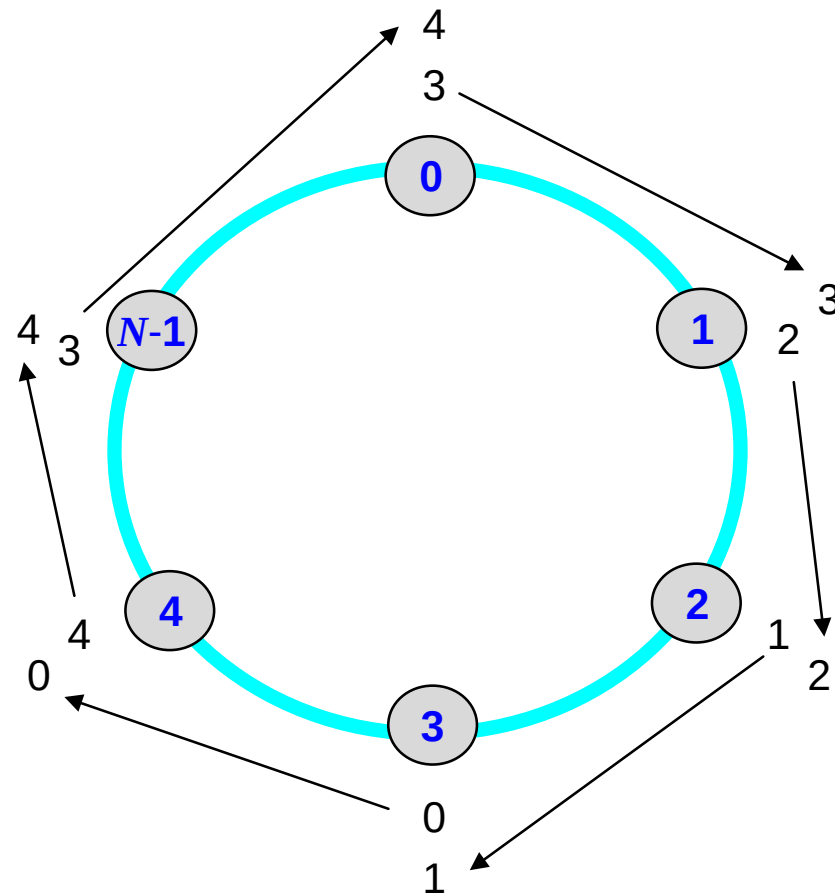


A worst-case failure scenario:

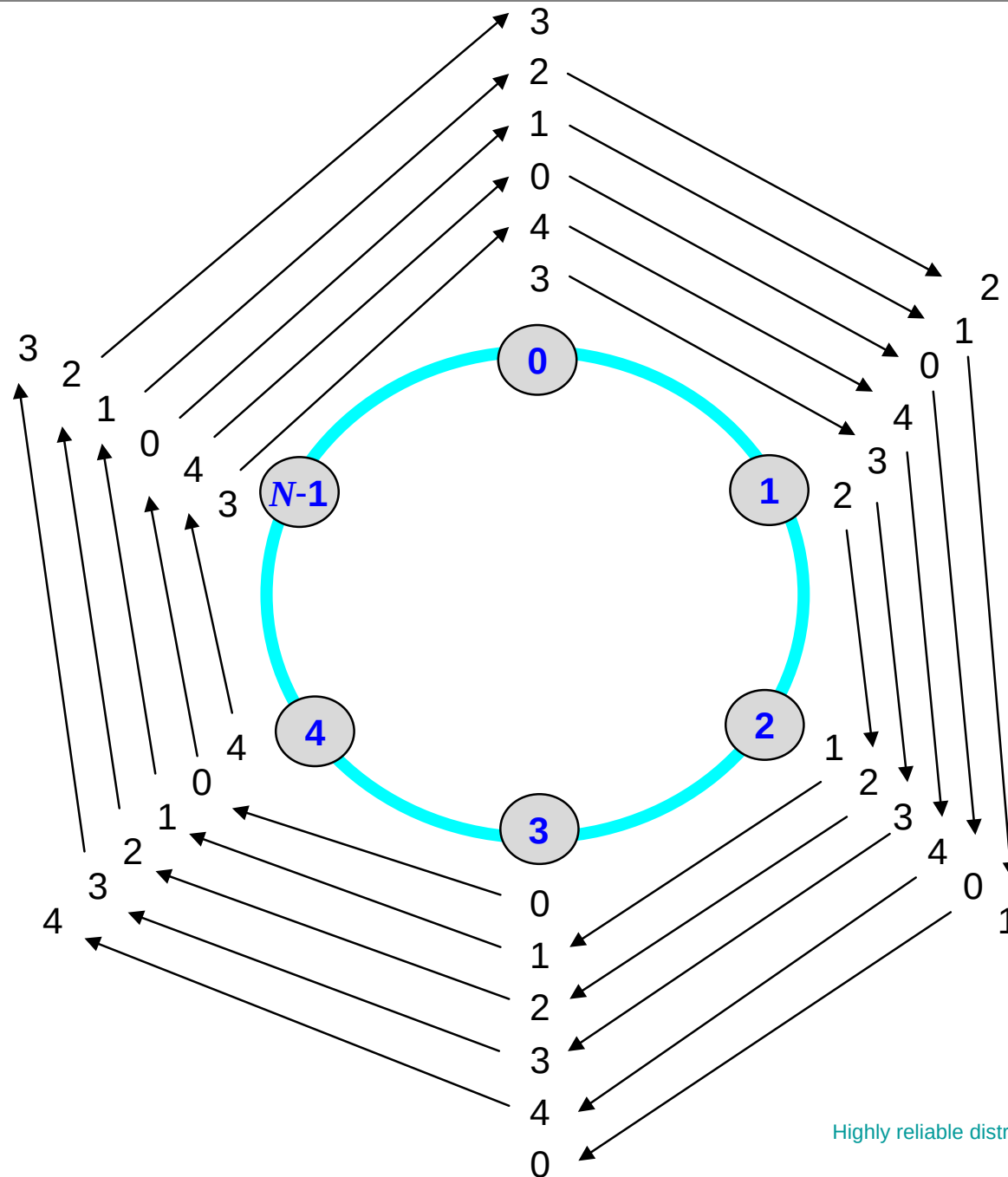
Let's start from:



DIJKSTRA'S MUTUAL EXCLUSION ALGORITHM (10)



(11)



Requirements

1. How the number of states can be bound?
 - $K \geq N$
2. What is the stabilization time?
 - This algorithm needs $\Omega(N^2)$ system steps before reaching a legal global state.

GENERAL PROPERTIES OF SELF-STABILIZING SYSTEMS/ALGORITHMS

The convergence to the subset of legal states provides a possibility of automatic recovery from any system failure after the causing fault has ceased, regardless of the state in which the system has been moved and regardless of quantity of the data has been corrupted. But the fault must be transient – it may change the state of the system, but not the system behavior.

Common advantages of self-stabilization:

- embedded tolerance of all **transient** faults
- no need of proper algorithm **initialization** (*the initial state need not to be legal*)
- tolerance of unreliability of communication channels and **dynamic** topology changes (*which can be perceived as **permanent** faults!*)

Drawbacks:

- acceptance of only **transient** faults
- possible **inconsistency** of system states during stabilization time
- no possibility of stabilization **detection**



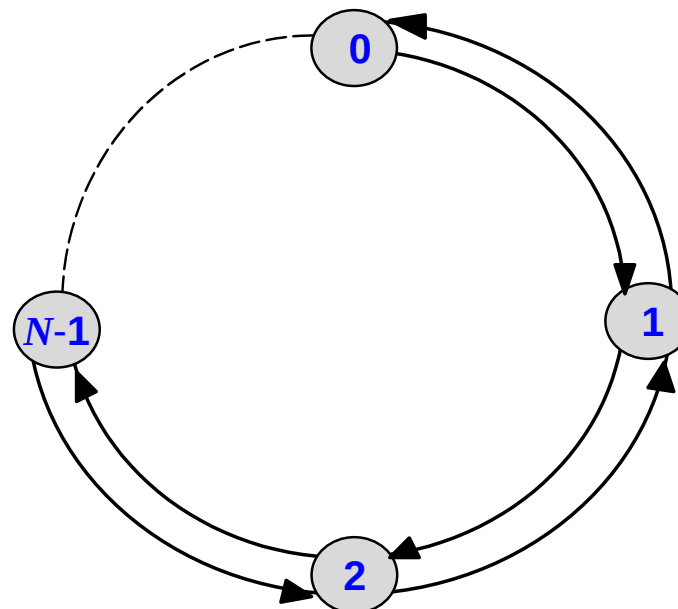
Different characteristics:

- **uniform** vs. **asymmetric** algorithms
- a number of states constant vs. depending on the size of the network ($K \geq N$)
- network **topology**
 - general topology: e.g. ring (unidirectional / bi-directional)
 - special topology
- **synchronous** (shared variables) communication
 - state reading model – any given process P_i can read in one step the entire state s_j of its neighbor P_j
 - link-registers model – communication is performed by distinct registers shared with different neighbors, thus P_i is able to transfer different data to each neighbor



Dijkstra's 3-state mutual exclusion (K=3)

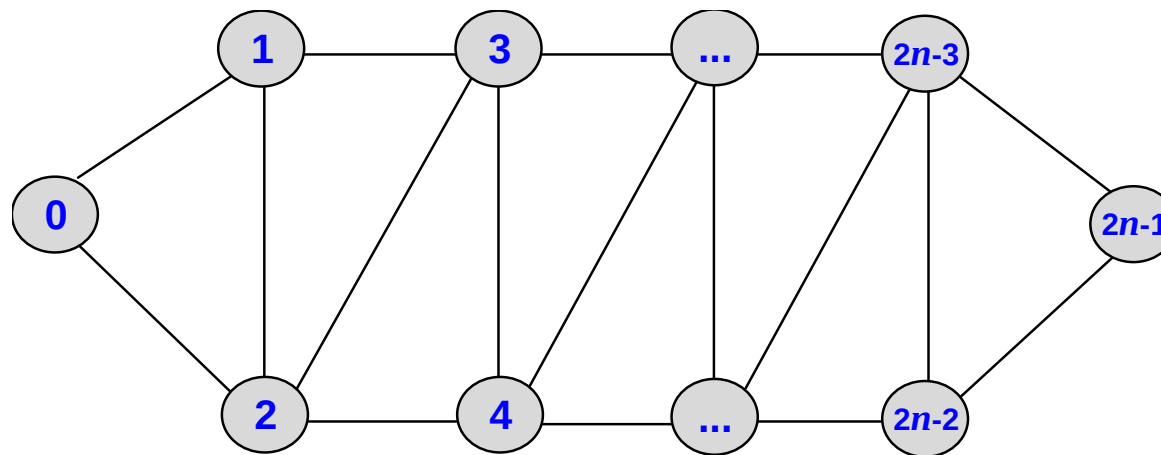
for P_0 :	if $(s_0+1)_{\text{mod } 3} = s_1$	then $s_0 = (s_0-1)_{\text{mod } 3}$
for P_{N-1} :	if $(s_{N-2}=s_0)$ AND $(s_{N-2}+1)_{\text{mod } 3} \neq s_{N-1}$	then $s_{N-1} = (s_{N-2} + 1)_{\text{mod } 3}$
for P_i :	OR if $(s_i+1)_{\text{mod } 3} = s_{i-1}$	then $s_i = s_{i-1}$
	if $(s_i+1)_{\text{mod } 3} = s_{i+1}$	then $s_i = s_{i+1}; 1 \leq i \leq N-2$





Ghosh's 2-state mutual exclusion ($K=2$)

- even number of processes: $N=2n$, where $n \geq 3$
- special topology:





Ghosh's 2-state mutual exclusion ($K=2$)

for P_0 :	if $s_0 \neq s_1$	then $s_0 = s_1$
for P_{2n-1} :	if $(s_{2n-1} = s_{2n-2})$	then $s_{2n-1} = \neg s_{2n-1}$
for even P_{2i} :	if $(s_{2i-2}, s_{2i-1}, s_{2i}, s_{2i+1}) = (x, x, \neg x, x)$	then $s_{2i} = x; 1 \leq i \leq n$
for odd P_{2i-1} :	if $(s_{2i-2}, s_{2i-1}, s_{2i}, s_{2i+1}) = (x, x, x, \neg x)$	then $s_{2i-1} = \neg x; 1 \leq i \leq n$

$\neg x = (1-x) \leftarrow \text{negacja binarna}$

DIFFERENT TYPES OF STABILIZATION

Let $(\mathbb{C}, \rightarrow)$ be a system.

Definition 3. A subset $\mathbb{P}$ of $\mathbb{C}$ is **strongly closed** iff for every transition $p \rightarrow q$, if $p \in \mathbb{P}$ then also $q \in \mathbb{P}$.

A subset $\mathbb{P}$ of $\mathbb{C}$ is **weakly closed** iff for every computation $(p_1, p_2, \dots, p_n, \dots)$ beginning in $\mathbb{P}$, there is a non-empty suffix where each state is in $\mathbb{P}$, i.e. if $p_1 \in \mathbb{P}$ then there exists $n \geq 1$, such that for all $i \geq n$: $p_i \in \mathbb{P}$.

- strong closure: $\forall p \rightarrow q : p \in \mathbb{P} \Rightarrow q \in \mathbb{P}$
- weak closure: $\forall p_1 \rightarrow p_2 \rightarrow \dots \rightarrow p_n \rightarrow \dots : p_1 \in \mathbb{P} \Rightarrow \exists n \geq 1 : p_n, p_{n+1}, \dots \in \mathbb{P}$

We can see that every strongly closed $\mathbb{P}$ is also weakly closed.



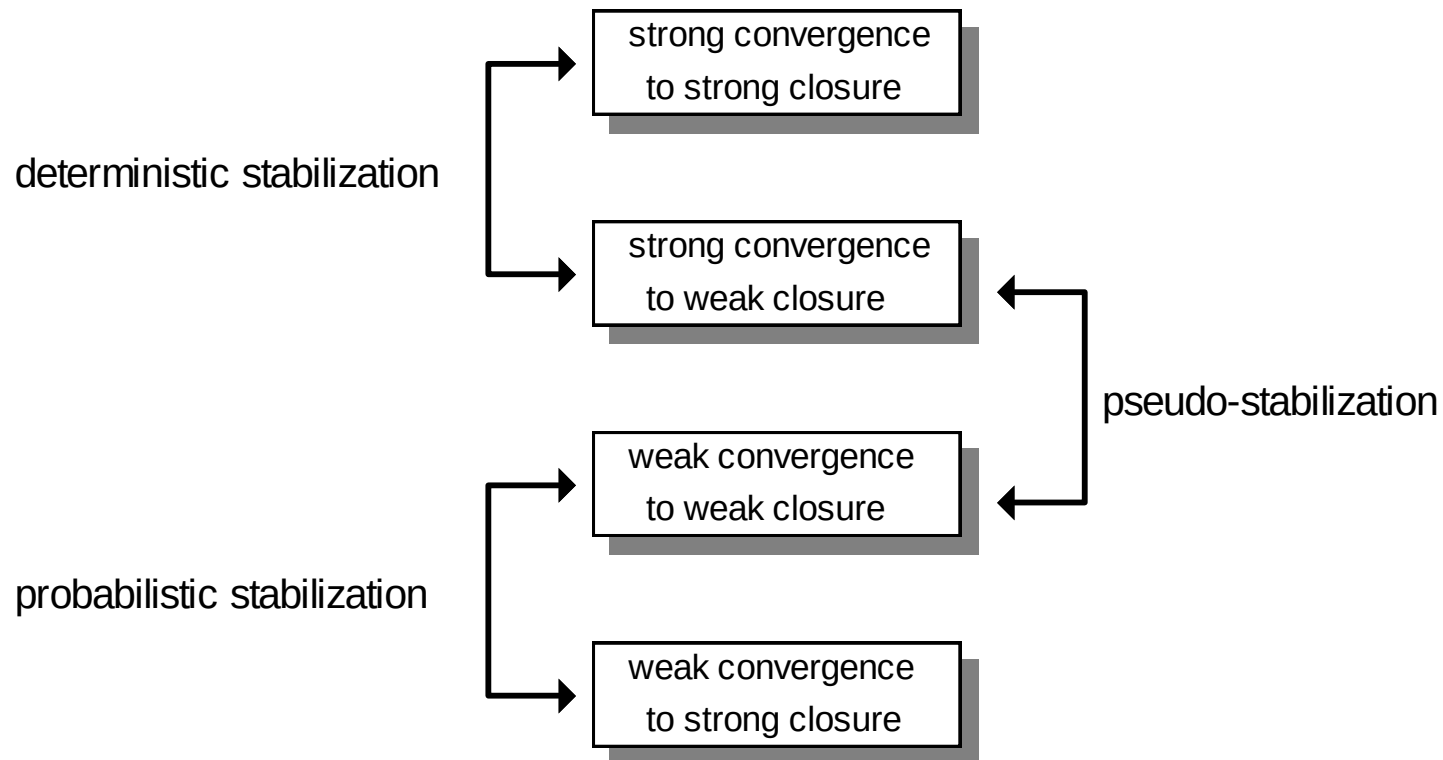
Now let $\mathbb{P}$ be a strongly closed subset of $\mathbb{C}$ and $\mathbb{Q}$ be a weakly (*perhaps also strongly*) closed subset of $\mathbb{C}$.

Definition 4. $\mathbb{P}$ **strongly converges** to $\mathbb{Q}$ iff every computation $(p_1, p_2, \dots)$ beginning in $\mathbb{P}$, has a state in $\mathbb{Q}$, i.e. if $p_1 \in \mathbb{P}$ then there exists $n \geq 1$, such that $p_n \in \mathbb{Q}$. $\mathbb{P}$ **weakly converges** to $\mathbb{Q}$ iff for every computation $(p_1, p_2, \dots)$ beginning in $\mathbb{P}$, for every state p_i in this computation there exists a computation $(p_i, q_2, q_3, \dots)$ beginning in this state p_i , which has a state in $\mathbb{Q}$, i.e. if $p_1 \in \mathbb{P}$ then for every $i \geq 1$, there exists a computation $(q_1, q_2, q_3, \dots)$ satisfying $q_1 = p_i$, and there exists $n \geq 1$, such that $q_n \in \mathbb{Q}$.

○ strong convergence: $\forall p_1 \rightarrow p_2 \rightarrow \dots \rightarrow p_n \rightarrow \dots : p_1 \in \mathbb{P} \Rightarrow \exists n \geq 1 : p_n \in \mathbb{Q}$

○ weak convergence: $\forall p_1 \rightarrow p_2 \rightarrow \dots \rightarrow p_n \rightarrow \dots : p_1 \in \mathbb{P} \Rightarrow$

$\forall n \geq 1 \exists p_n \rightarrow q_1 \rightarrow q_2 \rightarrow \dots \rightarrow q_i \rightarrow \dots : q_i \in \mathbb{Q}$



- **self-stabilization** = the subset $\mathbb{P}$ that converges to the set of legal states $\mathbb{L}$ is equal to the entire set $\mathbb{C}$ of all possible states
- **superstabilization** = stabilization + *passage* predicate, which is a specific safety property that has to be guaranteed while the system undergoes topology changes starting from a legal state



Laws of stabilization

$$\mathbb{P}, \mathbb{R} \subset \mathbb{C}.$$

Junctivity of closure:

- if both $\mathbb{P}$ and $\mathbb{R}$ are strongly closed then strongly closed are also $\mathbb{P} \cup \mathbb{R}$ and $\mathbb{P} \cap \mathbb{R}$.
- if both $\mathbb{P}$ and $\mathbb{R}$ are weakly closed then $\mathbb{P} \cup \mathbb{R}$ is weakly closed and $\mathbb{P} \cap \mathbb{R}$ is weakly closed (but $\mathbb{P} \cap \mathbb{R}$ may be strongly closed).

Junctivity of strong convergence:

- if $\mathbb{P}$ strongly converges to $\mathbb{Q}_P$ and $\mathbb{R}$ strongly converges to $\mathbb{Q}_R$ then $\mathbb{P} \cup \mathbb{R}$ strongly converges to $\mathbb{Q}_P \cup \mathbb{Q}_R$ and $\mathbb{P} \cap \mathbb{R}$ strongly converges to $\mathbb{Q}_P \cap \mathbb{Q}_R$.

Junctivity of strong stabilization:

- if $(\mathbb{C}, \rightarrow)$ strongly stabilizes to $\mathbb{P}$ and $(\mathbb{C}, \rightarrow)$ strongly stabilizes to $\mathbb{R}$ then $(\mathbb{C}, \rightarrow)$ strongly stabilizes to both $\mathbb{P} \cup \mathbb{R}$ and $\mathbb{P} \cap \mathbb{R}$.

Transitivity of strong convergence:

- if $\mathbb{P}$ strongly converges to $\mathbb{Q}$ and $\mathbb{Q}$ strongly converges to $\mathbb{R}$ then $\mathbb{P}$ strongly converges to $\mathbb{R}$.

Transitivity is a very useful lemma, enabling to prove stabilization of the whole system S , by showing stabilization of every subsystem of S .



Asymmetric	Uniform
mutual exclusion in token ring, N – arbitrary dining philosophers – deterministic	mutual exclusion in token ring, N – prime dining philosophers – probabilistic (no uniform deterministic solution exists)
Variable number of states	Constant number of states
unidirectional ring, deterministic, asymmetric: $K \geq N$	bi-directional ring, deterministic, asymmetric: $K=3$ special topology, deterministic, asymmetric: $K=2$ unidirectional ring, probabilistic: $K=2$
Deterministic	Probabilistic
dining philosophers – asymmetric bi-directional graph orientation – link registers, uniform	dining philosophers – uniform bi-directional graph orientation – asynchronous communication
Bounded stabilization time	Unbounded stabilization time
possible only within deterministic stabilization	pseudo-stabilization probabilistic stabilization

PROBABILISTIC STABILIZATION

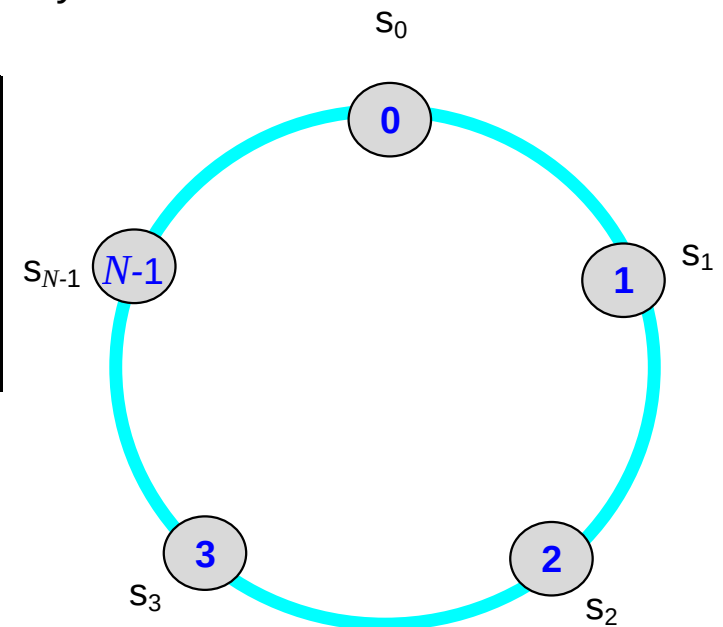
Herman's 2-state mutual exclusion ($K=2$)

- uniform algorithm
- odd number of processes in a unidirectional ring
- eventually only one privilege remains with probability 1

for P_i : $0 \leq i \leq N-1$

if $s_i \neq s_{i-1}$ then $s_i = s_{i-1}$

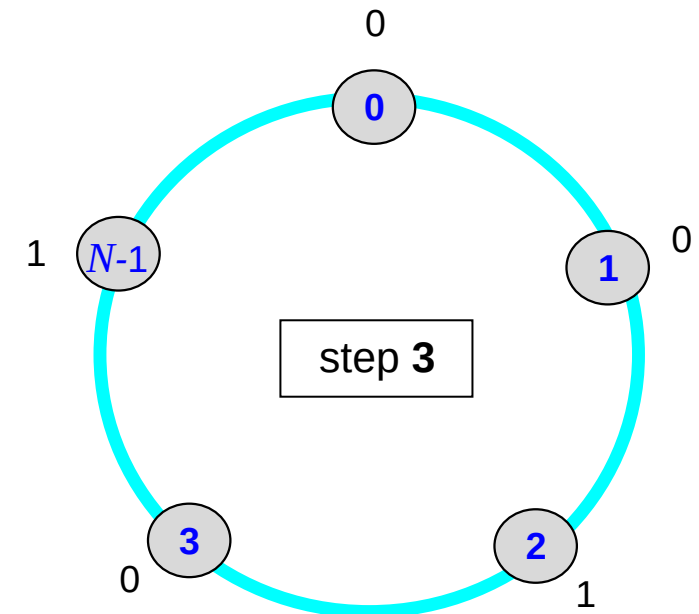
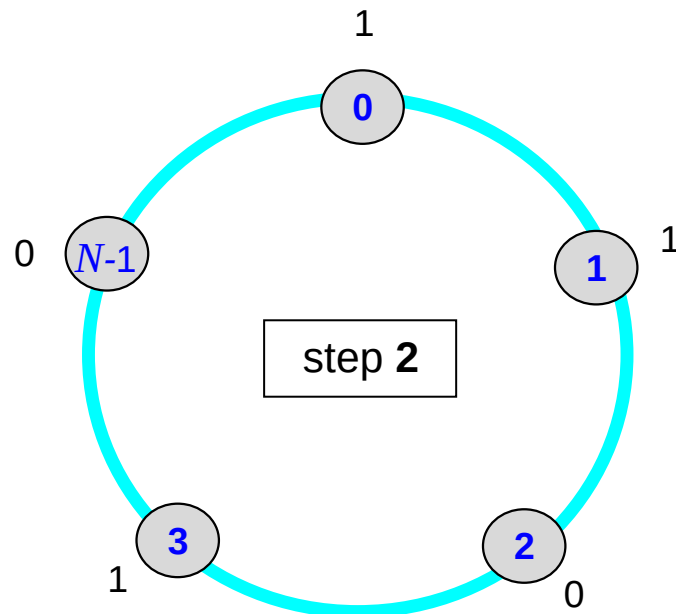
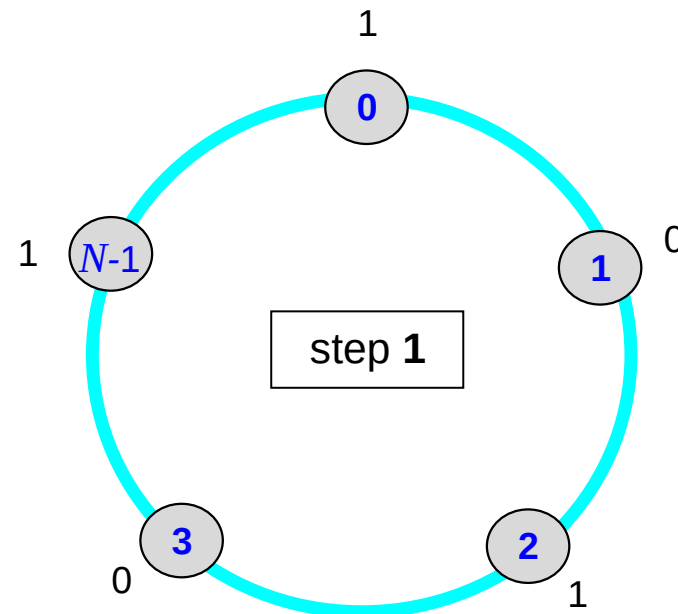
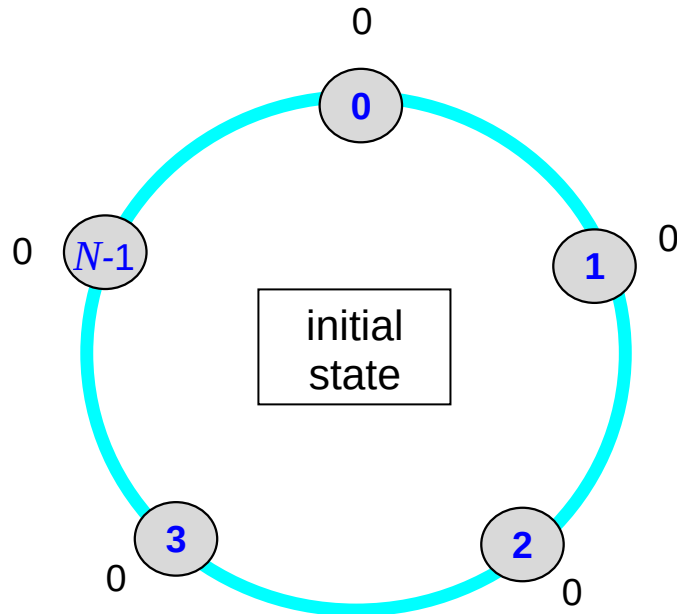
if $s_i = s_{i-1}$ then P_i has the privilege and $s_i = \text{random}(0;1)$



PROBABILISTIC STABILIZATION

(2)

INSTYTUT
INFORMATYKI



Leader election (Dolev et al.)

- algorithm starts with $D(N)$

$D(n)$:

{Termination condition}

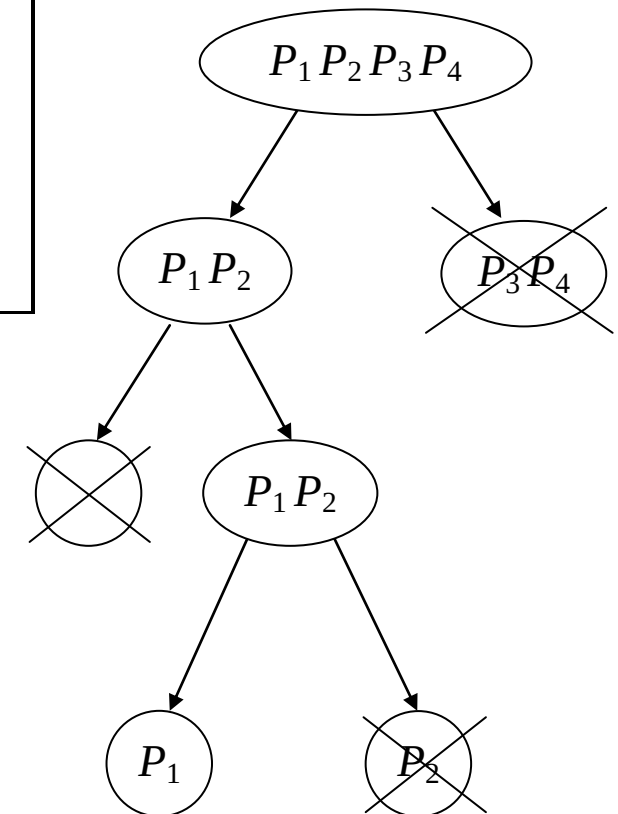
if $n = 1$ then Stop

{Randomized selection process}

if $n \geq 2$ then randomly divide n into $(n', n - n')$ (✖)

if $n' \neq 0$ then apply $D(n')$

else apply $D(n)$





Network medium access interpretation of leader election:

- used in random medium access for network stations
- during a time unit stations can detect either *silence*, *success*, or *collision*
- *silence* implies that no station tried to transmit a message
- *success* implies that only one station used the channel to transmit a message
- *collision* implies that at least two stations attempted to transmit messages

Distributed version of Dolev's algorithm:

- after flipping a coin ($\times$), there are two possibilities for station P_i :

Case 1: P_i tries to send the message, or

Case 2: P_i does not try to send the message

- in **Case 1**, there are two possible results:
 - *success* – then P_i is the leader = (trans-)mission accomplished
 - *collision* – P_i will flip a coin again
- in **Case 2**, there are three possibilities:
 - *silence* – then P_i will flip a coin again
 - *success* – P_i detects the leader
 - *collision* – P_i withdraws from the competition



Thus, the Dolev's algorithm can be rewritten as:

Station P_i flips a coin:

Case 1: P_i tries send the message

success: P_i is the leader

collision: P_i restarts (flips a coin again)

Case 2: P_i does not try to send the message

silence: P_i restarts (flips again the coin)

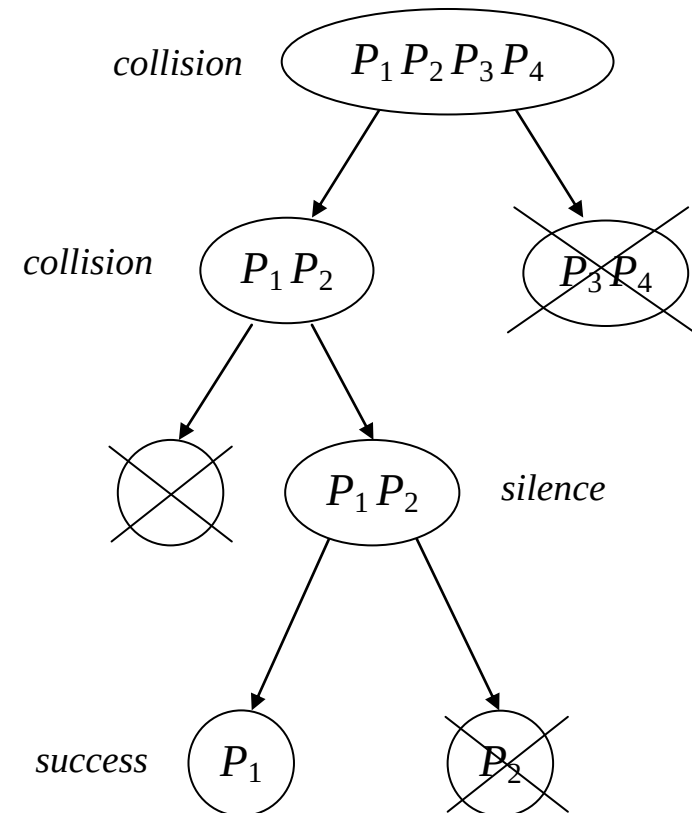
success: P_i detects the leader

collision: P_i is eliminated



Example:

- initially, in the first round, all the stations try to send their messages and a *collision* occurs
- in the next round, P_1 and P_2 send the messages again, while P_3 and P_4 do not participate
- because of both P_1 and P_2 , there is a *collision* again – as a result, P_3 and P_4 are eliminated
- now, in the next round, both P_1 and P_2 do not participate and the result is a *silence*
- so both of them flip a coin and now P_1 decides to send its message again
- and P_2 decides not to participate
- since P_1 is the only one sending a message
- the result is *success*
- and P_1 is the leader – the algorithm terminates



STABILIZATION IN NETWORKS

Many network oriented problems are actually graph problems:

- network reconfiguration
- graph orientation
- shortest path
- spanning-tree construction
- ...

They appear often in:

- routing protocols
- STP in transparent bridging

STP in Ethernet (IEEE 802.1d)

If a network segment becomes unreachable or network parameters are changed, the protocol automatically reconfigures the spanning-tree topology by activating some standby paths.

- initially, each switch believes it is the root of the spanning tree
- switches regularly exchange status information:
 - the identifier of the transmitting switch (usually a MAC address)
 - the identifier of the switch which is believed to be the root of the tree
 - the “cost” of the path towards the root.
- a given switch uses this information to choose the shortest path towards the current root
- if there are multiple possible roots, it selects the root with the smallest identifier (lowest MAC address)
- links that are not included in the spanning tree are placed in blocking mode (do not forward packets, but still transport status information)

BIBLIOGRAPHY

- [1]  G. Tel, "Introduction to Distributed Algorithms", Cambridge University Press, 2000, ch.17.
- [2]  Ajay D. Kshemkalyani, Mukesh Singhal, "Distributed Computing. Principles, Algorithms, and Systems", Cambridge University Press, 2008, ch. 17.
- [3]  Roger Wattenhofer, "Mastering Distributed Algorithms", Inverted Forest Publishing, 2020, ch. 11.
- [4]  Jerzy Brzeziński, Michał Szychowiak, "Self-Stabilization in Distributed Systems—a Short Survey", *Foundations of Computing and Decision Sciences*, vol. 25, no. 1, 2000.
(<https://www.cs.put.poznan.pl/mszychowiak/publications/FCDS-25-1-2000.pdf>)
- [5]  M. Flatebo, A. K. Datta, S. Ghosh, "Self Stabilization in Distributed Systems", in "Readings in Distributed Computing Systems", IEEE Computer Society Press, 1994, pp.100-114.
- [6]  M. G. Gouda, "The Triumph and Tribulation of System Stabilization", in "Distributed Algorithms", *Lecture Notes in Computer Science*, vol. 972, Springer, 1995, pp.1-18.